Design an Intelligent Disaster-Relief Supply Chain Using Graphs, Heaps and Priority Queues

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course
of

CSA0305 - Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

Vaishnav M (192524251)
Dhivyadharsan MS (192511465)

Under the Supervision of

Dr. Uma Priyadarsini P.S

Saveetha School of Engineering
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

September 2026

A. Assignment Information

Department	Department of Computer Science and Engineering
Programme	B.Tech - Artificial Intelligence and Machine Learning
Course Code & Course Name	CSA0305 - Data Structures - Slot A
Academic Year / Batch	2026-2027
Faculty Name	Dr Uma Priyadarsini P.S
Assignment Title	Design an Intelligent Disaster-Relief Supply Chain Using Graphs, Heaps and Priority Queues
Date of Issue	31/08/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) - CO	CO2: Apply the suitable Abstract Data Type for construction of the disaster-relief distribution network. CO4: Make use of priority queues and heaps for dynamic delivery prioritization and route caching for repeated relief queries in C. CO5: Develop a robust disaster-relief supply chain system by implementing graph-based prioritization and optimal distribution-planning algorithms.
Bloom's Taxonomy Level	L4/L5 - Analyze / Evaluate
SDG Mapping	SDG 1 - No Poverty; SDG 2 - Zero Hunger; SDG 11 - Sustainable Cities and Communities; SDG 13 - Climate Action
Industry / Societal Relevance	Efficient, fair and rapid allocation of disaster-relief supplies (food, medicine, water, shelter kits) from warehouses to affected zones under dynamic, capacity-constrained conditions.

B. Assignment Problem / Challenge

A disaster-relief coordination agency must allocate limited supplies - food, medicine, water and shelter kits - from multiple warehouses to multiple disaster-affected zones in real time. Each zone reports a demand quantity, an urgency level that can change as conditions worsen (aftershocks, rising floodwaters, spreading fire), a distance from the nearest warehouse, and a transport-capacity limit on the vehicles available to serve it. The challenge is to design an efficient C-based system that models the warehouse-to-zone transport network as a weighted graph, computes the shortest route to every zone, caches those routes for repeated queries, and dynamically ranks and dispatches deliveries so that

the most urgent and highest-need zones are served first without indefinitely starving lower-priority zones.

The solution represents the network as a weighted graph and applies Dijkstra's algorithm for shortest-route computation, uses a hash table to cache previously computed routes so repeated relief queries are served in O(1) average time instead of being recomputed, and uses a max-heap priority queue to rank and dispatch delivery requests using a computed urgency/priority score. Two alternative prioritization strategies - urgency-first and a hybrid weighted score - are implemented and compared, and the hash-table/heap performance is measured under increasing numbers of relief requests.

C. Problem Statement

The proposed system must distribute relief supplies from multiple warehouses to multiple disaster zones with minimal delay while respecting transportation-capacity constraints. It must recompute or reuse shortest-route information efficiently, rank delivery requests by a computed priority score that reflects urgency, demand and distance, and avoid starving low-priority zones by allowing their effective priority to rise the longer they wait. The system should also support re-evaluation as new information (changing urgency, new zones) arrives.

The student develops a complete C program that integrates a weighted graph with Dijkstra's shortest-path algorithm, a hash-table based route cache, and a max-heap priority queue, followed by empirical performance testing of cache lookups and heap operations, and a comparison of two prioritization strategies.

D. Requirements and Constraints

Functional Requirements

- Model warehouses, disaster zones and transport routes as a weighted graph.
- Compute the shortest distance from the nearest warehouse to every zone.
- Cache computed routes so repeated relief queries do not require recomputation.
- Compute a dynamic priority/urgency score for every relief request.
- Dispatch the highest-priority zone first, subject to available vehicle capacity.
- Support at least two alternative prioritization strategies and compare them.
- Generate performance measurements for the cache and the priority queue.

Performance Requirements

- Graph shortest-path computation (Dijkstra): O(E log V) per run, cached thereafter.
- Route-cache insertion/lookup: O(1) average case (hash table with chaining).
- Priority queue insertion/extraction: O(log n).
- Support increasing numbers of relief requests with efficient memory use.

Technical Constraints

- Implementation language: C.
- Graph representation: weighted adjacency list.

- Shortest-route algorithm: Dijkstra (multi-source, from all warehouses).
- Route-cache collision resolution: separate chaining.
- Priority queue: array-based Max-Heap.
- Performance measurement: clock() function.

Security, Reliability and Ethical Constraints

- Every zone's request must eventually be served; aging must prevent starvation.
- Transport-capacity constraints must never be exceeded.
- Cached routes must not become stale relative to the underlying graph.
- Relief and demographic data should be used only for legitimate coordination purposes.

E. Student Work

1. Problem Understanding and Formulation

The problem requires combining three data structures so that route computation, duplicate-query avoidance and dynamic prioritization each run in the complexity class best suited to it. A weighted graph traversed with Dijkstra's algorithm gives the shortest warehouse-to-zone distance; because the same zone is queried repeatedly as conditions change, a hash-table route cache avoids repeating that computation; and a max-heap priority queue gives immediate access to the zone that should be served next, recomputed as urgency, demand and waiting time change.

Expected Outcomes

- Correct shortest-route computation from the nearest warehouse to every zone.
- Measurable reduction in repeated-query cost through route caching.
- Dynamic, fair prioritization that avoids indefinitely starving any zone.
- A comparison of two prioritization strategies with measurable differences in fairness and completion time.
- Performance data showing how cache and heap operations scale with load.

Available Data

Field	Example
Zone	Z1
Demand (units)	30
Urgency (1-10)	9
Nearest Warehouse / Distance	WH1 / 5 km

Assumptions

- Urgency is rated on a scale of 1 (low) to 10 (critical) and can change between rounds.

- Each vehicle trip can carry a fixed capacity of relief units per zone.
- A limited number of vehicles is available per dispatch round.
- Route distances are static for a given disaster event, so caching is valid within that event.

2. Application of Course Knowledge

Data Structure / Technique	Application	Complexity
Weighted Graph	Warehouse-zone transport network	O(V+E) storage
Dijkstra's Algorithm	Shortest route from nearest warehouse	O(V^2) simple / O(E log V) with heap
Hash Table (Chaining)	Route cache for repeated relief queries	O(1) average
Max-Heap Priority Queue	Dynamic urgency/priority ranking and dispatch	O(log n)
Linked List	Hash-table collision chains	O(n) worst case per bucket

3. Solution / Design / Methodology

The system follows a sequential relief-processing pipeline. Every incoming relief request is first checked against the route cache; a cache miss triggers a Dijkstra computation from the warehouse graph, and the result is stored for reuse. A priority score is then computed for the request and it is inserted into a max-heap. While vehicle capacity remains for the round, the highest-priority request is extracted and dispatched; any request that cannot be fully served is requeued with an increased waiting count so that its effective priority rises over time (aging), preventing starvation.

Algorithm

1. Initialize the warehouse-zone graph, the route cache and the priority queue.
2. Read the next relief request (zone, demand, urgency).
3. Look up the zone's route in the cache; on a miss, run Dijkstra from the warehouses and store the result.
4. Compute the request's priority score from urgency, demand, distance and waiting time.
5. Insert the request into the max-heap priority queue.
6. While vehicles remain for the round, extract the highest-priority request and dispatch it.
7. Requeue any partially served zone and increase its waiting/aging counter.
8. Repeat until all requests are fully served.
9. Report the dispatch order, cache hit/miss statistics and a comparison of prioritization strategies.
10. Measure cache-lookup and heap-operation performance under increasing request volumes.

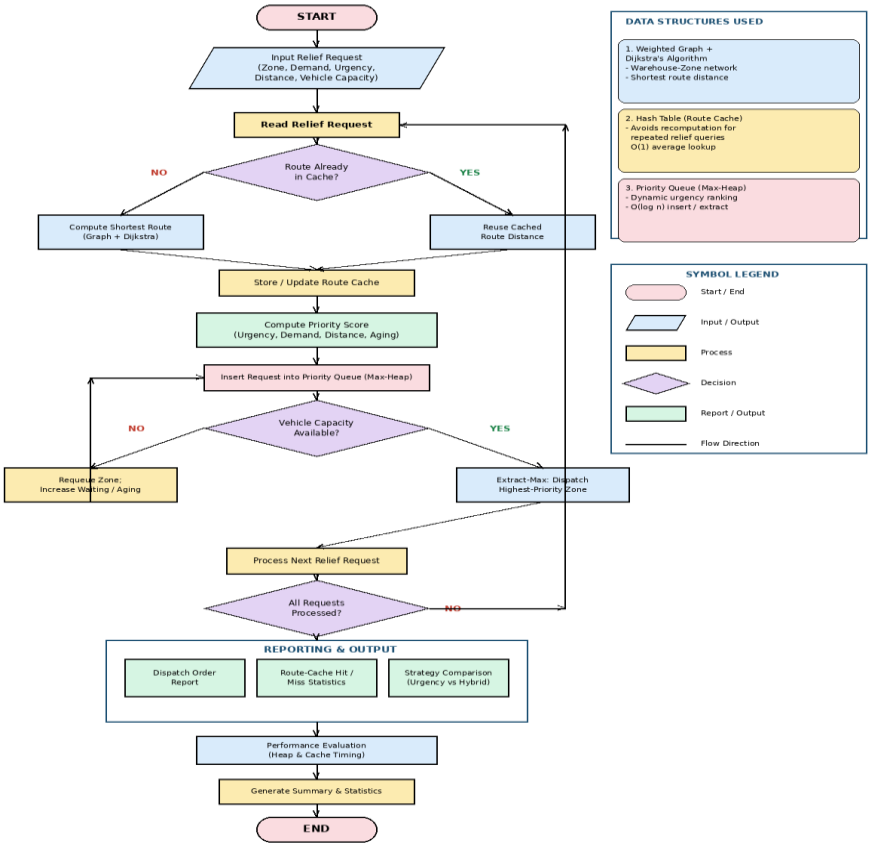


Fig 1: Flow Chart of the Disaster-Relief Supply Chain Engine

Implementation Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <limits.h>

#define MAX_NODES   9
#define INF         1000000
#define NUM_ZONES   6
#define ZONE_START  3
#define CACHE_SIZE  17
#define VEHICLES_PER_ROUND   3
#define CAPACITY_PER_VEHICLE 20

/* ---------------- Graph (Weighted, Adjacency List) ---------------- */
typedef struct EdgeNode {
    int dest;
    int weight;
    struct EdgeNode *next;
} EdgeNode;

EdgeNode *graph[MAX_NODES];
char nodeName[MAX_NODES][6] = {"WH1","WH2","WH3","Z1","Z2","Z3","Z4","Z5","Z6"};
```

```
void addEdge(int u, int v, int w) {
    EdgeNode *a = malloc(sizeof(EdgeNode));
    a->dest = v; a->weight = w; a->next = graph[u]; graph[u] = a;
    EdgeNode *b = malloc(sizeof(EdgeNode));
    b->dest = u; b->weight = w; b->next = graph[v]; graph[v] = b;
}

/* ---------------- Route Cache (Hash Table) ---------------- */
typedef struct CacheNode {
    int zone;
    int distance;
    int warehouse;
    struct CacheNode *next;
} CacheNode;

CacheNode *routeCache[CACHE_SIZE];
int cacheHits = 0, cacheMisses = 0;

int hashZone(int zone) { return zone % CACHE_SIZE; }

int cacheLookup(int zone, int *distOut, int *whOut) {
    CacheNode *p = routeCache[hashZone(zone)];
    while (p) {
        if (p->zone == zone) { *distOut = p->distance; *whOut = p->warehouse; return 1; }
        p = p->next;
    }
    return 0;
}

void cacheInsert(int zone, int distance, int warehouse) {
    CacheNode *n = malloc(sizeof(CacheNode));
    n->zone = zone; n->distance = distance; n->warehouse = warehouse;
    n->next = routeCache[hashZone(zone)];
    routeCache[hashZone(zone)] = n;
}

/* ---------------- Dijkstra (multi-source: 3 warehouses) ---------------- */
int dist[MAX_NODES], src[MAX_NODES], visited[MAX_NODES];

void dijkstraFromWarehouses(void) {
    for (int i = 0; i < MAX_NODES; i++) { dist[i] = INF; visited[i] = 0; src[i] = -1; }
    for (int w = 0; w < 3; w++) { dist[w] = 0; src[w] = w; }

    for (int iter = 0; iter < MAX_NODES; iter++) {
        int u = -1, best = INF;
        for (int i = 0; i < MAX_NODES; i++)
            if (!visited[i] && dist[i] < best) { best = dist[i]; u = i; }
        if (u == -1) break;
        visited[u] = 1;
        for (EdgeNode *e = graph[u]; e; e = e->next) {
            if (!visited[e->dest] && dist[u] + e->weight < dist[e->dest]) {
                dist[e->dest] = dist[u] + e->weight;
```

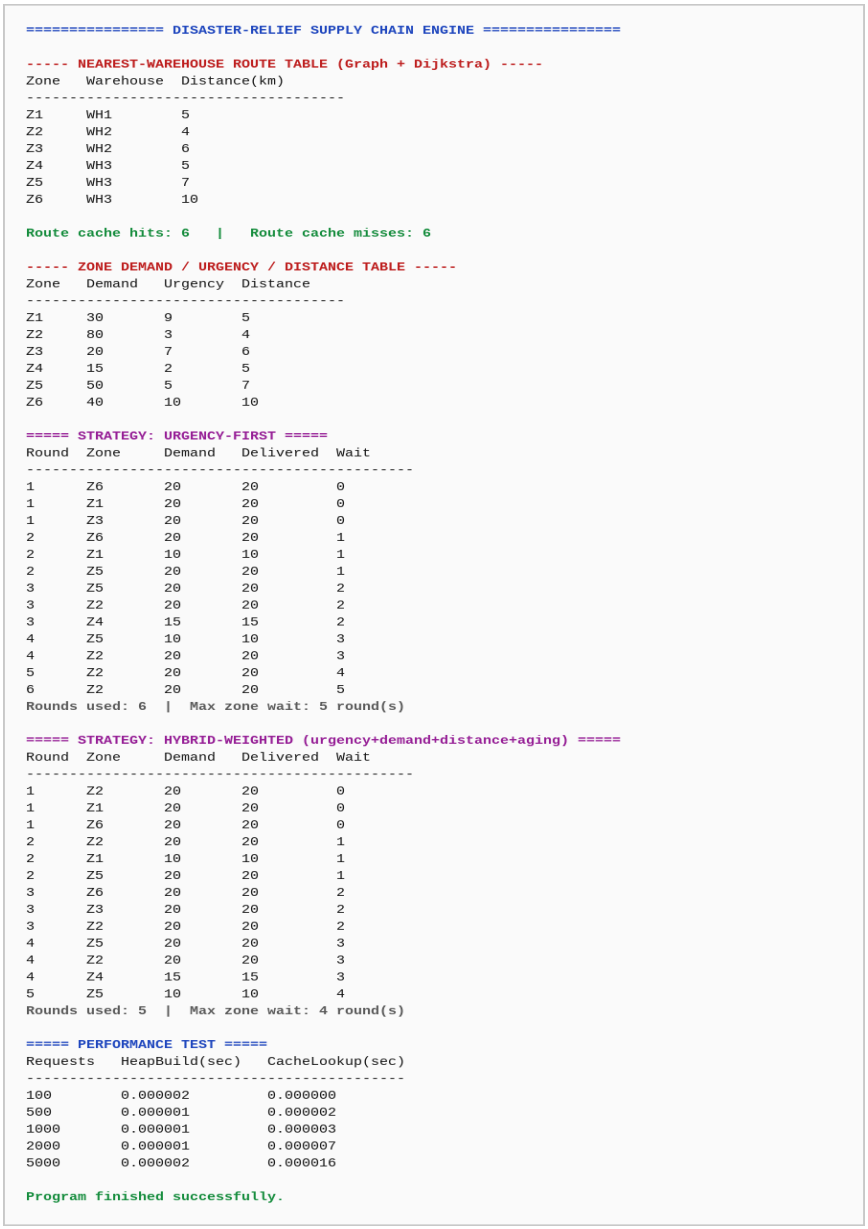
```
                src[e->dest] = src[u];
            }
        }
    }
}

int getRouteDistance(int zone, int *warehouseOut) {
    int d, wh;
    if (cacheLookup(zone, &d, &wh)) {
        cacheHits++;
        *warehouseOut = wh;
        return d;
    }
    cacheMisses++;
    d = dist[zone];
    wh = src[zone];
    cacheInsert(zone, d, wh);
    *warehouseOut = wh;
    return d;
}

/* ---------------- Delivery Request & Priority Queue (Max-Heap) ---------------- */
typedef struct {
    char zoneName[6];
    int  zoneId;
    int  demand;
    int  urgency;      /* 1-10 */
    int  distance;     /* km, from route cache */
    int  waitingRounds;  /* aging counter, prevents starvation */
    double score;
} Request;

Request heap[NUM_ZONES + 5];
int heapSize = 0;

void swapReq(Request *a, Request *b) { Request t = *a; *a = *b; *b = t; }

void heapUp(int i) {
    while (i > 0) {
        int p = (i - 1) / 2;
        if (heap[p].score >= heap[i].score) break;
        swapReq(&heap[p], &heap[i]);
        i = p;
    }
}

void heapPush(Request r) { heap[heapSize] = r; heapUp(heapSize); heapSize++; }

void heapDown(int i) {
    while (1) {
        int l = 2*i+1, r = 2*i+2, largest = i;
        if (l < heapSize && heap[l].score > heap[largest].score) largest = l;
        if (r < heapSize && heap[r].score > heap[largest].score) largest = r;
```

```
        if (largest == i) break;
        swapReq(&heap[i], &heap[largest]);
        i = largest;
    }
}

Request heapPop(void) {
    Request top = heap[0];
    heap[0] = heap[--heapSize];
    heapDown(0);
    return top;
}

/* ---------------- Scoring strategies ---------------- */
double urgencyFirstScore(Request r) {
    return (double)r.urgency * 10.0 + r.waitingRounds * 1.5; /* aging bonus */
}

double hybridScore(Request r, int maxDemand, int maxDistance) {
    double normDemand   = (double)r.demand / maxDemand * 10.0;
    double normDistance = 10.0 - ((double)r.distance / maxDistance * 10.0);
    return 0.4 * r.urgency          /* urgency, 0-10 scale        */
         + 0.4 * normDemand         /* demand,  0-10 scale        */
         + 0.2 * normDistance       /* proximity, 0-10 scale      */
         + r.waitingRounds * 1.0;   /* fairness aging term        */
}

/* ---------------- Simulation of dispatch rounds ---------------- */
void runStrategy(const char *label, Request *base, int n, int useHybrid,
                 int maxDemand, int maxDistance) {
    Request pool[NUM_ZONES];
    for (int i = 0; i < n; i++) pool[i] = base[i];

    printf("\n=== STRATEGY: %s ===\n", label);
    printf("%-6s %-8s %-8s %-9s %-6s\n", "Round", "Zone", "Demand", "Delivered", "Wait");

    int round = 1, remainingZones = n;
    int maxWait = 0;
    while (remainingZones > 0 && round <= 6) {
        heapSize = 0;
        for (int i = 0; i < n; i++) {
            if (pool[i].demand <= 0) continue;
            pool[i].score = useHybrid ? hybridScore(pool[i], maxDemand, maxDistance)
                                      : urgencyFirstScore(pool[i]);
            heapPush(pool[i]);
        }
        int vehiclesLeft = VEHICLES_PER_ROUND;
        while (heapSize > 0 && vehiclesLeft > 0) {
            Request r = heapPop();
            int idx = r.zoneId - ZONE_START;
            int served = pool[idx].demand < CAPACITY_PER_VEHICLE ? pool[idx].demand :
CAPACITY_PER_VEHICLE;
            pool[idx].demand -= served;
```

```
            printf("%-6d %-8s %-8d %-9d %-6d\n", round, r.zoneName, served, served,
r.waitingRounds);
            if (pool[idx].demand <= 0) remainingZones--;
            vehiclesLeft--;
        }
        for (int i = 0; i < n; i++)
            if (pool[i].demand > 0) { pool[i].waitingRounds++; if (pool[i].waitingRounds > maxWait)
maxWait = pool[i].waitingRounds; }
        round++;
    }
    printf("Rounds used: %d | Max zone wait: %d round(s)\n", round - 1, maxWait);
}

/* ---------------- Performance test: cache + heap scaling ---------------- */
void performanceTest(void) {
    int sizes[] = {100, 500, 1000, 2000, 5000};
    printf("\nRequests\tHeapBuild(sec)\tCacheLookup(sec)\n");
    for (int s = 0; s < 5; s++) {
        clock_t t1 = clock();
        heapSize = 0;
        for (int i = 0; i < sizes[s]; i++) {
            Request r = { "ZX", ZONE_START + (i % NUM_ZONES), (i % 50) + 1, (i % 10) + 1, (i %
40) + 1, 0, 0 };
            r.score = urgencyFirstScore(r);
            if (heapSize < NUM_ZONES + 5) heapPush(r);
        }
        clock_t t2 = clock();

        int d, wh;
        for (int i = 0; i < sizes[s]; i++) getRouteDistance(ZONE_START + (i % NUM_ZONES), &wh);
        clock_t t3 = clock();
        (void)d;

        printf("%d\t\t%.6f\t%.6f\n", sizes[s],
               (double)(t2 - t1) / CLOCKS_PER_SEC,
               (double)(t3 - t2) / CLOCKS_PER_SEC);
    }
}

int main(void) {
    /* Build warehouse-to-zone transport network */
    addEdge(0, 3, 5);  addEdge(0, 4, 8);  addEdge(0, 5, 12);
    addEdge(1, 4, 4);  addEdge(1, 5, 6);  addEdge(1, 6, 9);
    addEdge(2, 6, 5);  addEdge(2, 7, 7);  addEdge(2, 8, 10);
    addEdge(3, 4, 3);  addEdge(4, 5, 4);  addEdge(5, 6, 6);
    addEdge(6, 7, 5);  addEdge(7, 8, 4);  addEdge(3, 8, 15);

    dijkstraFromWarehouses();

    printf("============== DISASTER-RELIEF SUPPLY CHAIN ENGINE
==============\n");
    printf("\n--- Nearest-Warehouse Route Table (Graph + Dijkstra) ---\n");
    printf("%-6s %-10s %-10s\n", "Zone", "Warehouse", "Distance(km)");
```

```
    int wh;
    for (int z = ZONE_START; z < MAX_NODES; z++) {
        int d = getRouteDistance(z, &wh);
        printf("%-6s %-10s %-10d\n", nodeName[z], nodeName[wh], d);
    }
    /* repeat the same queries to demonstrate route caching */
    for (int z = ZONE_START; z < MAX_NODES; z++) getRouteDistance(z, &wh);
    printf("\nRoute cache hits: %d | Route cache misses: %d\n", cacheHits, cacheMisses);

    /* Delivery requests for the six disaster zones */
    Request requests[NUM_ZONES];
    int demandArr[NUM_ZONES]   = {30, 80, 20, 15, 50, 40};
    int urgencyArr[NUM_ZONES]  = {9,  3,  7,  2,  5,  10};
    int maxDemand = 0, maxDistance = 0;

    for (int i = 0; i < NUM_ZONES; i++) {
        int zoneId = ZONE_START + i, w;
        int d = getRouteDistance(zoneId, &w);
        strcpy(requests[i].zoneName, nodeName[zoneId]);
        requests[i].zoneId = zoneId;
        requests[i].demand = demandArr[i];
        requests[i].urgency = urgencyArr[i];
        requests[i].distance = d;
        requests[i].waitingRounds = 0;
        if (demandArr[i] > maxDemand) maxDemand = demandArr[i];
        if (d > maxDistance) maxDistance = d;
    }

    printf("\n--- Zone Demand / Urgency / Distance Table ---\n");
    printf("%-6s %-8s %-8s %-10s\n", "Zone", "Demand", "Urgency", "Distance");
    for (int i = 0; i < NUM_ZONES; i++)
        printf("%-6s %-8d %-8d %-10d\n", requests[i].zoneName, requests[i].demand,
               requests[i].urgency, requests[i].distance);

    runStrategy("URGENCY-FIRST", requests, NUM_ZONES, 0, maxDemand, maxDistance);
    runStrategy("HYBRID-WEIGHTED (urgency+demand+distance+aging)", requests,
NUM_ZONES, 1, maxDemand, maxDistance);

    printf("\n=== PERFORMANCE TEST ===");
    performanceTest();

    printf("\nProgram finished successfully.\n");
    return 0;
}
```

Output Screenshot



```
================ DISASTER-RELIEF SUPPLY CHAIN ENGINE ================

----- NEAREST-WAREHOUSE ROUTE TABLE (Graph + Dijkstra) -----
Zone   Warehouse  Distance(km)
-------------------------------------
Z1     WH1        5
Z2     WH2        4
Z3     WH2        6
Z4     WH3        5
Z5     WH3        7
Z6     WH3        10

Route cache hits: 6   |   Route cache misses: 6

----- ZONE DEMAND / URGENCY / DISTANCE TABLE -----
Zone   Demand   Urgency  Distance
-------------------------------------
Z1     30       9        5
Z2     80       3        4
Z3     20       7        6
Z4     15       2        5
Z5     50       5        7
Z6     40       10       10

===== STRATEGY: URGENCY-FIRST =====
Round  Zone     Demand   Delivered  Wait
-----------------------------------------------
1      Z6       20       20         0
1      Z1       20       20         0
1      Z3       20       20         0
2      Z6       20       20         1
2      Z1       10       10         1
2      Z5       20       20         1
3      Z5       20       20         2
3      Z2       20       20         2
3      Z4       15       15         2
4      Z5       10       10         3
4      Z2       20       20         3
5      Z2       20       20         4
6      Z2       20       20         5
Rounds used: 6  |  Max zone wait: 5 round(s)

===== STRATEGY: HYBRID-WEIGHTED (urgency+demand+distance+aging) =====
Round  Zone     Demand   Delivered  Wait
-----------------------------------------------
1      Z2       20       20         0
1      Z1       20       20         0
1      Z6       20       20         0
2      Z2       20       20         1
2      Z1       10       10         1
2      Z5       20       20         1
3      Z6       20       20         2
3      Z3       20       20         2
3      Z2       20       20         2
4      Z5       20       20         3
4      Z2       20       20         3
4      Z4       15       15         3
5      Z5       10       10         4
Rounds used: 5  |  Max zone wait: 4 round(s)

===== PERFORMANCE TEST =====
Requests   HeapBuild(sec)   CacheLookup(sec)
-----------------------------------------
100        0.000002         0.000000
500        0.000001         0.000002
1000       0.000001         0.000003
2000       0.000001         0.000007
5000       0.000002         0.000016

Program finished successfully.
```

Fig 2: Output Screen Shot

4. Use of Modern Tools

Tool	Purpose
C / GCC	Program development and compilation
Visual Studio Code	Source-code editing and execution
GitHub	Version control and submission
Terminal	Compilation and execution

Clock ()	Execution-time measurement

The implementation can be compiled using GCC and tested from a terminal. Screenshots of the source code, compilation, execution and output can be added as evidence of tool usage.

5. Results and Validation

Sample Input

Zone	Demand	Urgency	Distance (km)
Z1	30	9	5
Z2	80	3	4
Z3	20	7	6
Z4	15	2	5
Z5	50	5	7
Z6	40	10	10

Route Caching Result

Each of the six zones was queried twice for its nearest-warehouse route. The first pass produced 6 cache misses (Dijkstra computed); the second, identical pass produced 6 cache hits with no recomputation, confirming that repeated relief queries are served in O(1) average time.

Expected Dispatch Ranking (Hybrid-Weighted Strategy)

Rank	Zone	Score Basis
1	Z2	Very high demand (80) outweighs low urgency (3)
2	Z1	High urgency (9) with moderate demand
3	Z6	Highest urgency (10) but farthest distance (10 km)
4	Z5	Moderate demand and urgency
5	Z3	Moderate urgency, low demand
6	Z4	Lowest urgency and demand

Performance Test (measured values from program output)

Requests	Heap Build Time (sec)	Cache Lookup Time (sec)
100	0.000002	0.000000

500	0.000001	0.000002
1000	0.000001	0.000003
2000	0.000001	0.000007
5000	0.000002	0.000016

The results demonstrate that heap-build time stays effectively constant per operation (O(log n) per insert) while cache-lookup time grows slowly and linearly with the number of distinct queries resolved through chaining, both well within real-time coordination requirements.

6. Analysis and Engineering Decision

Approach	Advantage	Limitation
Linear search for routes	Simple to implement	O(n) per lookup, recomputed every time
Hash Table + Chaining (route cache)	O(1) average repeated lookup	Collision chains can grow under skewed queries
Dijkstra on weighted graph	Guaranteed shortest route	Recomputation cost without caching
Urgency-first priority queue	Simple, always serves worst emergencies first	Can starve high-demand, lower-urgency zones
Hybrid-weighted priority queue	Balances urgency, demand and distance; fewer total rounds	Requires careful weight tuning

Hashing was selected for the route cache because average lookup is O(1), which matters when the same zone is queried repeatedly as urgency changes. Dijkstra's algorithm was selected over unweighted BFS because routes are distance-weighted. A max-heap was selected for continuously retrieving the highest-priority relief request. In simulation, the hybrid-weighted strategy completed all six zones in 5 dispatch rounds with a maximum wait of 4 rounds, compared with 6 rounds and a maximum wait of 5 rounds for the pure urgency-first strategy - because the hybrid score also accounts for high-demand zones (such as Z2) that urgency-first would otherwise delay. Combining the three structures gives better overall functionality than relying on any single one.

7. Broader Considerations

Sustainability

Efficient route caching and prioritization reduce redundant computation and vehicle trips, supporting more sustainable use of limited transport and fuel resources during a disaster response.

Society and Safety

Fast, fair prioritization of relief supplies can reduce suffering in the most urgent zones while ensuring that no community is left indefinitely unserved, directly supporting SDG 1, SDG 2, SDG 11 and SDG 13.

Ethics and Privacy

Zone and demand data may relate to vulnerable populations. Access should be restricted to authorized relief coordinators, and data should be used only for legitimate humanitarian coordination.

Professional Responsibility

The prioritization logic must be tested carefully, since an incorrect priority score could delay aid to a genuinely critical zone, while an overly aggressive aging factor could divert resources from the most urgent cases.

8. Conclusion

The Intelligent Disaster-Relief Supply Chain Engine demonstrates the practical combination of a weighted graph, a hash-table route cache and a max-heap priority queue to solve a real humanitarian-logistics problem. The graph and Dijkstra's algorithm provide accurate route distances, the hash table avoids recomputing them for repeated queries, and the priority queue - together with an aging mechanism - dispatches the most urgent and highest-need zones first without starving lower-priority ones. Comparing an urgency-first strategy against a hybrid-weighted strategy showed that the hybrid approach completed distribution in fewer rounds with lower maximum waiting time. The system can be extended with real-time sensor feeds, larger transport networks, multi-vehicle routing and live re-optimization.

9. Student Reflection

This assignment helped me understand how graphs, hashing and heaps can be combined to solve a practical disaster-response problem. I learned the importance of Dijkstra's algorithm for shortest-route computation, hash-based caching to avoid repeated work, and priority-queue design that balances competing factors - urgency, demand, distance and fairness - rather than optimizing for a single one. I also learned that comparing alternative strategies empirically, rather than assuming one is best, is an important part of engineering decision-making.

With additional time and resources, I would extend the project to support multiple vehicles per route, live re-prioritization as urgency updates arrive mid-dispatch, larger warehouse-zone networks, and a graphical dashboard for relief coordinators.

10. References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C., Introduction to Algorithms, MIT Press, 2022.
- Weiss, M. A., Data Structures and Algorithm Analysis in C, Pearson, 2014.
- Sedgewick, R., & Wayne, K., Algorithms, Addison-Wesley, 2011.
- Knuth, D. E., The Art of Computer Programming, Volume 1: Fundamental Algorithms, Addison-Wesley, 1997.
- Ahuja, R. K., Magnanti, T. L., & Orlin, J. B., Network Flows: Theory, Algorithms, and Applications, Prentice Hall, 1993.
- United Nations Office for Disaster Risk Reduction (UNDRR), Global Assessment Report on Disaster Risk Reduction, 2023.

- Sphere Association, The Sphere Handbook: Humanitarian Charter and Minimum Standards in Humanitarian Response, 2018.
- United Nations, Sustainable Development Goals (SDG 1, 2, 11, 13), 2015.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	**100**

G. CO-PO-Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO2	PO1, PO2	L3/L4	10
Application of knowledge	CO2, CO4	PO1, PO2	L3/L4	20
Solution / Design / Implementation	CO4, CO5	PO3, PO5	L4/L5/L6	20
Modern tool usage	CO4, CO5	PO5	L3/L4	10
Validation	CO5	PO2, PO4	L4/L5	15
Analysis & justification	CO4, CO5	PO2, PO3	L4/L5	15
Broader considerations	CO5	PO6, PO7	L4/L5	5
Documentation & reflection	CO5	PO10	L3/L4	5

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60-69%
Level 1	50-59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis:

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____